Session 1: Introduction to R

Objects, Data, Functions, and Reproducible Workflows

Karry Jiao
dj398@cam.ac.uk
KJ Github
Oct. 2026

Table of contents

1. Introduction
2. Lecture review
3. Initial settings
4. File paths and projects
5. Objects and data structures
6. Working with data frames
7. Functions
8. Data transformation
9. Reproducible workflows
10. Appendix

Introduction

What this session is about

This session introduces a single mental model for learning R:

> **R stores things as objects. Objects have a type and structure. Functions take objects as inputs and return results, which can become new objects.**

By the end, you should be able to:

- recognize the main structures in which R stores data;
- inspect, select, filter, and transform a data frame;
- understand functions, arguments, and returned values; and
- organize code so that an analysis is reproducible.

Lecture Review

Lecture Review — How data can mislead

- **Stocks and flows answer different questions.** A flow records change during a period; a stock records an accumulated level at a point in time.
- **The denominator defines a rate.** Always ask how a reported percentage was calculated.
- **Absolute and relative values tell different stories.** The interpretation depends on the baseline and comparison.
- **Conditional probabilities cannot be reversed.**

$$P(\text{working family} \mid \text{poverty}) \neq P(\text{poverty} \mid \text{working family})$$

Lecture Review — Baselines and aggregation

Relative change

The baseline is the denominator:

$$\text{Relative change} = \frac{P_2 - P_1}{P_1}$$

The same absolute change can imply very different relative changes.

Aggregate mean

Group size determines the weight:

$$\bar{x} = \frac{\sum_i n_i \bar{x}_i}{\sum_i n_i}$$

An unweighted average of group means can give a different answer.

Lecture Review — Population and sample

- A **parameter** describes a population; a **statistic** is calculated from a sample.
- Sample statistics estimate population parameters.
- With appropriate random sampling, the sample mean can be unbiased: $E(\bar{x}) = \mu$.
- Larger random samples generally reduce sampling variation, but they do not repair systematic bias.

$$\text{Estimate} - \text{Truth} = (\text{Estimate} - E[\text{Estimate}]) + (E[\text{Estimate}] - \text{Truth})$$

Lecture Review — Describing a distribution

Centre and position

- mean
- median
- quartiles
- minimum and maximum

Spread and shape

- standard deviation
- interquartile range
- skewness
- unusual observations

Always inspect the distribution: similar averages can conceal very different spreads and shapes.

Lecture Review — Statistical notation

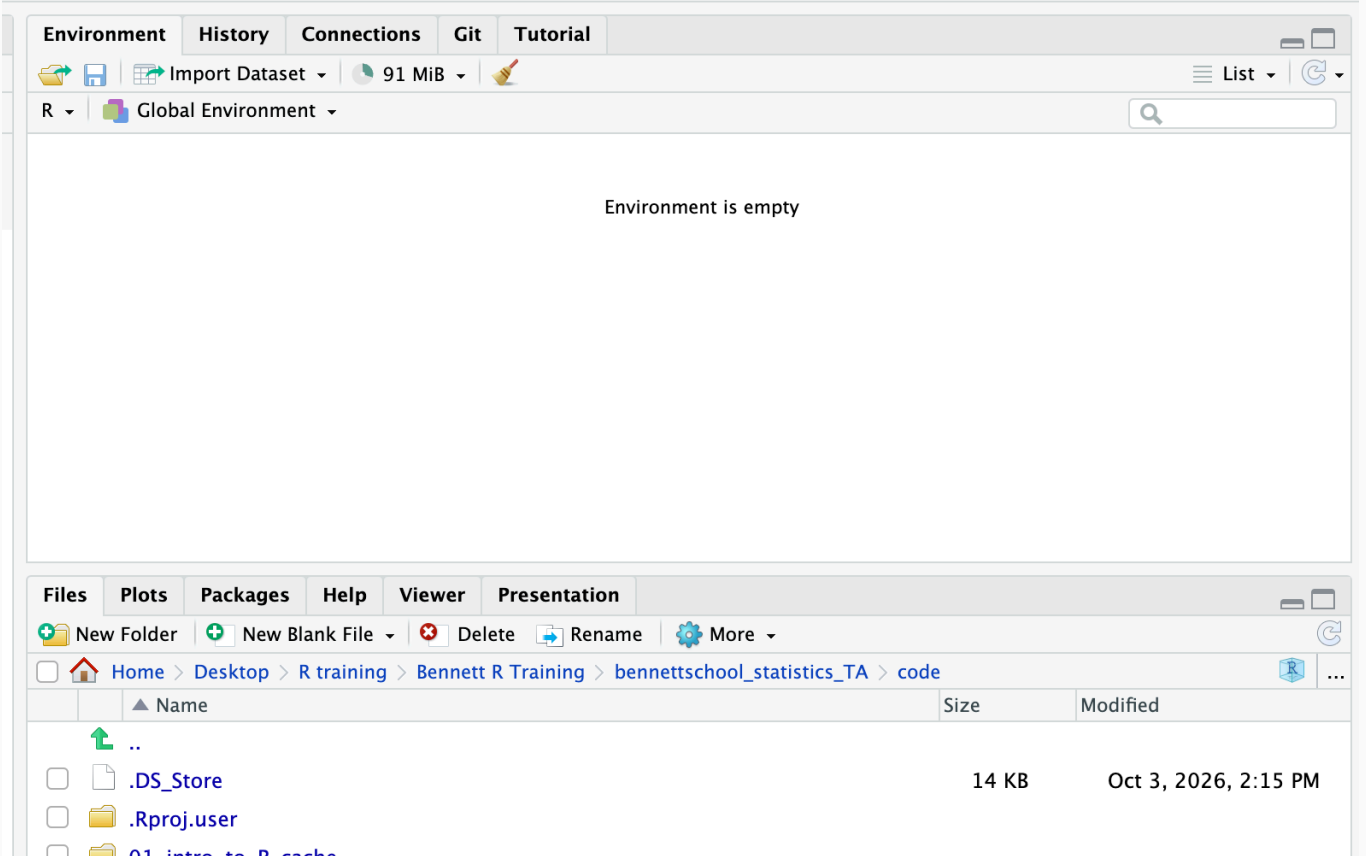
Population quantity	Symbol	Sample analogue	Symbol
Mean	μ	Mean	$\bar{x}$
Variance	σ^2	Variance	s^2
Standard deviation	σ	Standard deviation	s
Proportion	π	Proportion	$\hat{p}$
Correlation	ρ	Correlation	r
Regression slope	β	Estimated slope	$\hat{\beta}$

Initial settings

Initial settings

- Let's start by opening RStudio or by closing and opening it again
- Notice two things:

    1. Your environment is *probably* empty (it's okay if it's not)

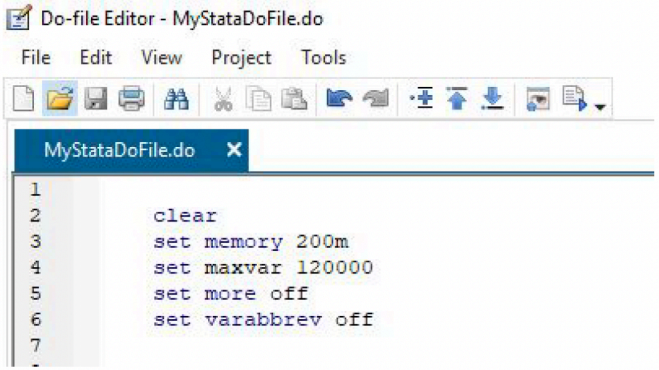


Initial settings

- Let's start by opening RStudio or by closing and opening it again
- Notice two things:
    1. Your environment is *probably* empty (it's OK if it's not)
    2. Go to the `Console` panel and use the up and down keys to navigate through previously executed commands. They are saved by default in a file named `.Rhistory`.

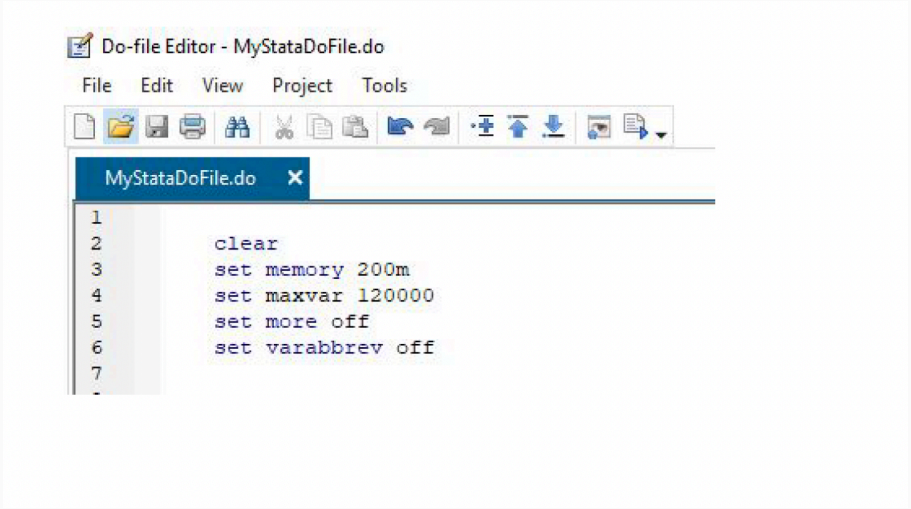
Initial settings

Have you ever seen these lines of code before in STATA?



Initial settings

Have you ever seen these lines of code before?



- We **don't need to set the memory or the maximum number of variables** in R
- The equivalent of `set more off` is the default
- The equivalent of `clear` in R is `rm(list = ls())`, which removes all the objects in the environment. It is not always default setting, but we'll make sure it is set in exercise 1

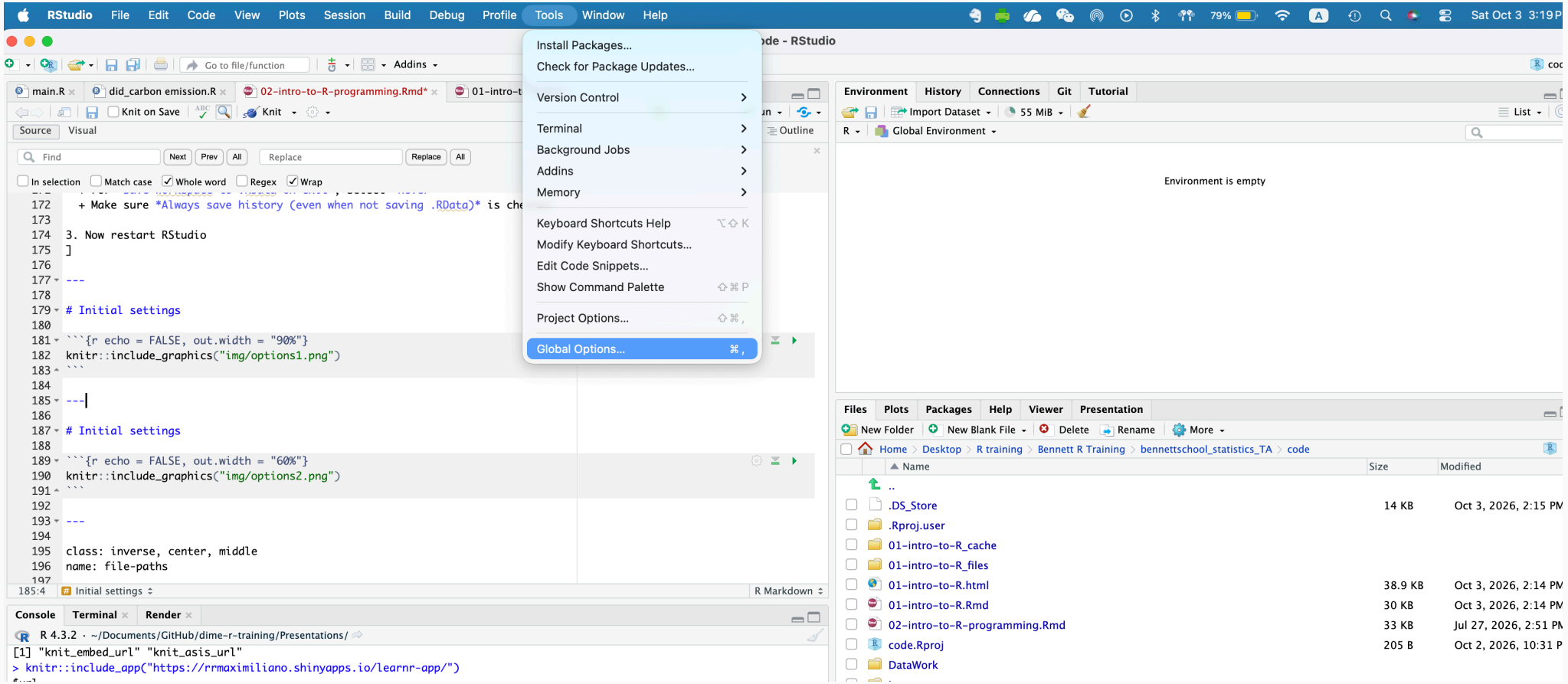
Initial settings

Exercise 1 ⌨ (🕓 1 min)

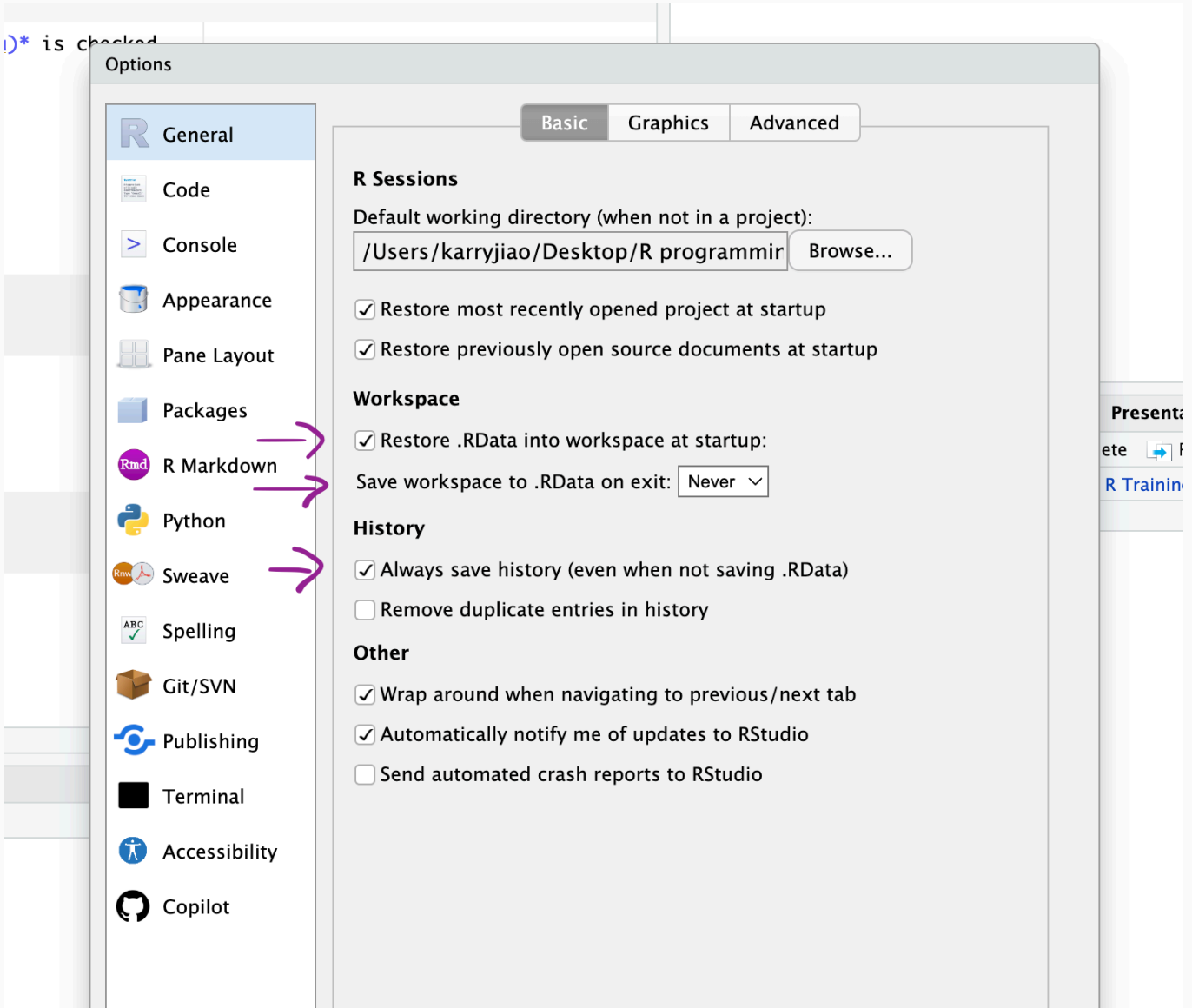
After this, you'll never have to use the equivalent of `clear all`

1. Go to `Tools` > `Global Options` ...
2. In the `General` tab, make sure the following options are set:
    - Un-check *Restore .RData into workspace at startup*
    - For *Save workspace to .RData on exit*, select *Never*
    - Make sure *Always save history (even when not saving .RData)* is checked
3. Now restart RStudio

Initial settings



Initial settings



Initial settings — `.Rhistory` and `.RData`

- `.Rhistory` can store commands previously entered in the Console.
- `.RData` can save objects from the Environment and restore them in a later session.
- Both are normally associated with the working directory in which the R session started.

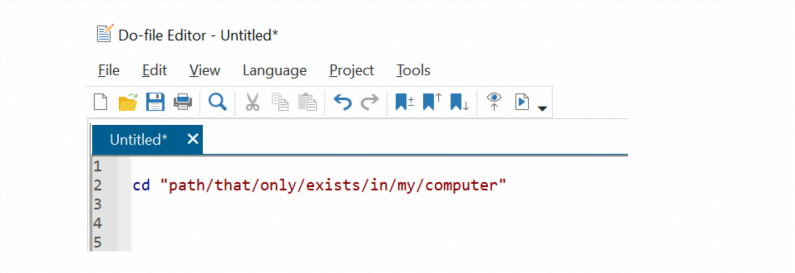
Recommended practice

- Do **not** restore `.RData` automatically at startup.
- Do **not** automatically save the workspace on exit.
- Recreate important objects by running a saved script.

File paths

File paths

- What about working directories? We usually do something like this every time we start a new script in Stata:



- The command to print the current working directory in R is:

```
getwd()
```

- And the direct equivalent to `cd` in R is this command:

```
setwd("your/path")
```

- However, we recommend not using it unless it's absolutely necessary (never, if possible)

RStudio projects

- Instead, you should use RStudio projects and the `here` library
- RStudio projects let you "bind" your project files to a root directory, regardless of the path to it
- This is crucial because it allows smooth interoperability between different computers where the exact path to the project root directory differs
- Additionally, each RStudio project you work on keeps their own history of commands!

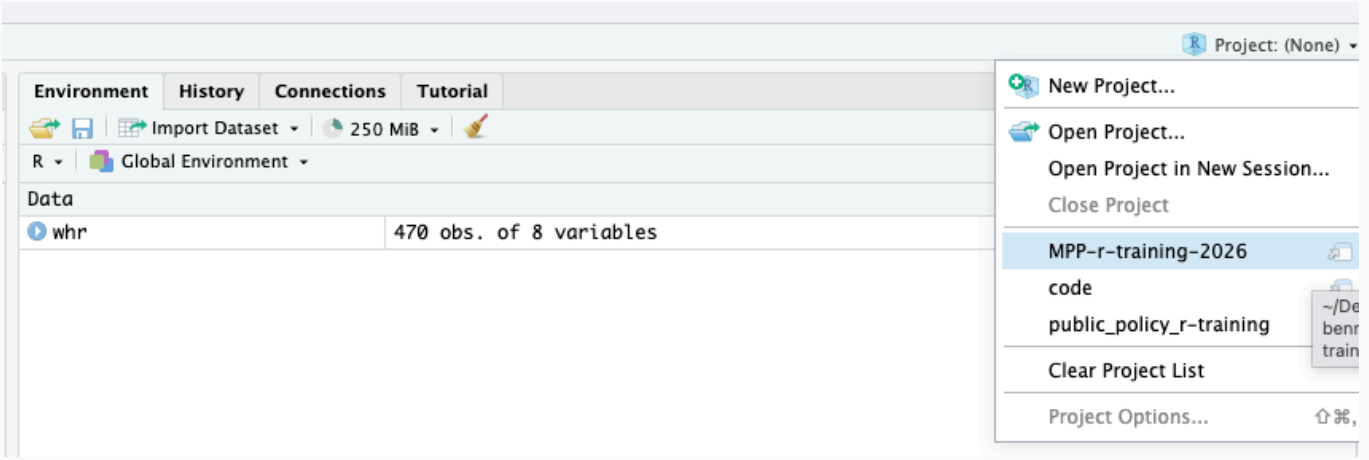
Important: We won't get into the specifics of directory organization here, but we'll assume that all the files you use for a specific project (data, scripts, and outputs) reside in the same project directory. We'll call this the **working directory**.

RStudio projects

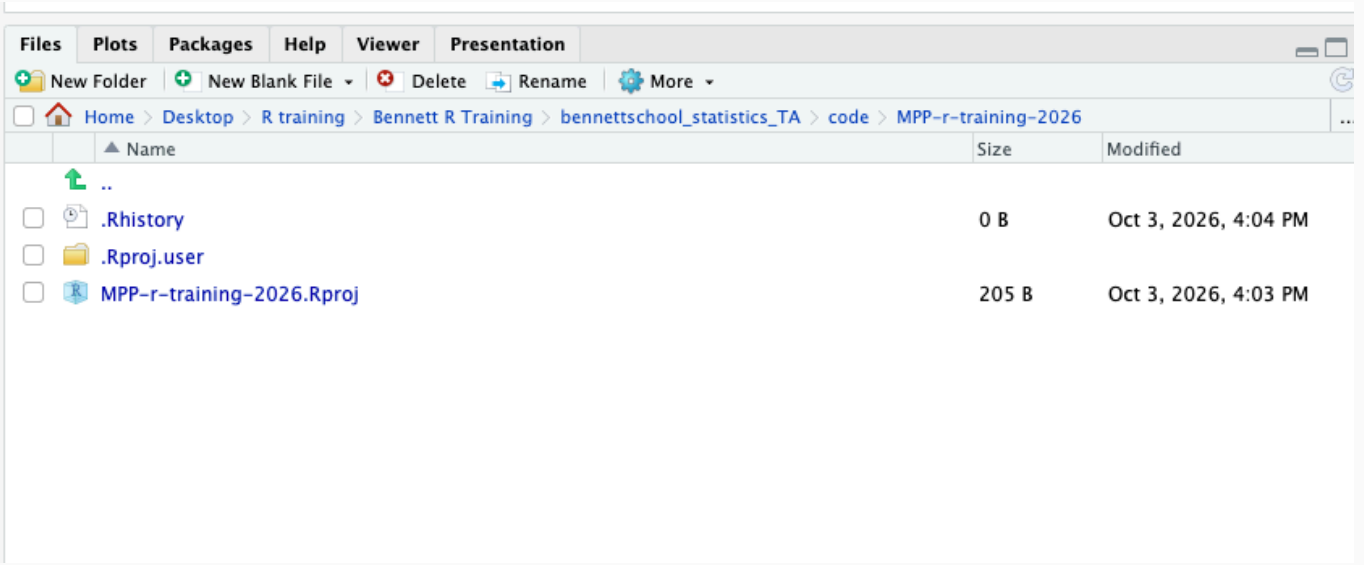
Exercise 2 ⌨ (🕒 3 min)

1. Create a folder named `MPP-r-training-2026` in your preferred location in your computer
2. Go to https://github.com/karryjiaodonghua/MPP_statistics_session and download the file `DataWork.zip`
3. Unzip `DataWork.zip` in your folder `MPP-r-training-2026`
4. On RStudio, select `File` > `New Project ...` (the window will load for a few seconds)
5. Select `Existing Directory`
6. Browse to the location of `MPP-r-training-2026` and select `Create Project`

RStudio projects



RStudio projects



The `here` library

Now that we're working in a project, we can use the library `here` to define any file paths relative to the project folder.

- `here` locates files relative to your project root
- It uses the root project directory to build paths to files easily
- It allows for interoperability between different computers where the absolute path to the same file is not the same

Usage of `here`

- Load `here`

```
install.packages("here") # install first if you don't have it
library(here)
```

- Now you'll be able to use `here()` to point the location of every file relative to your project root
    - For example, to load a `csv` file located in: `C:/Desktop/MPP-r-training-2016/DataWork/Dataset/Raw/data-file.csv`, you should use:

```
path ← here("DataWork", "Dataset", "Raw","data-file.csv")
df   ← read.csv(path)
```

- **Notes:**
    - Your project root is the directory that contains the `.Rproj` file
    - The result of `here()` is an absolute path that points to a file or folder location in your computer

File paths

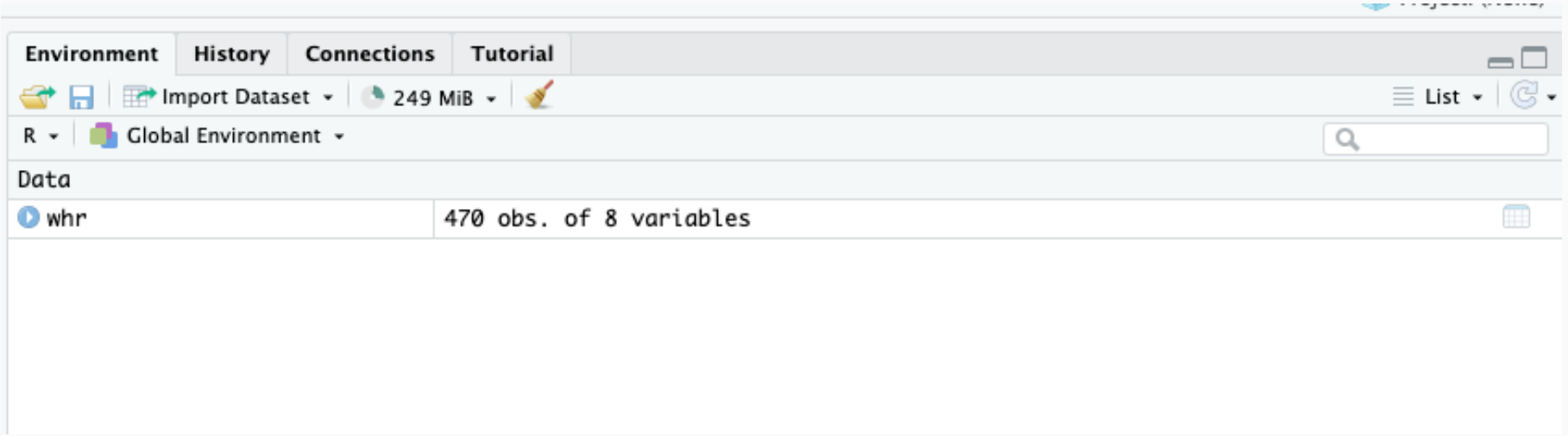
Exercise 3 ⌨ (🕓 3 min)

1. Go to `File` > `New File` > `R Script` to open a new script
2. In the new script, load `here` and read the `.csv` file in `DataWork/DataSets/Final/whr_panel.csv` using `here()`
   - Use the function `read.csv()` to load the file. The argument for `read.csv()` is the result of `here()`
   - Remember to assign the dataframe you're reading to an object with `←`. You can call it `whr` as I asked you to download the dataset from the website directly.
   - If you get an error saying `could not find function "here"`, then load the library first with `library(here)`

```
whr ← read.csv(here("DataWork", "DataSets", "Final", "whr_panel.csv"))
```

RStudio projects and `here`

If you did the exercise correctly, you should see the `whr` dataframe listed in the Environment panel



RStudio projects — Further reading

RStudio Projects keep an analysis anchored to a stable project root and make relative paths portable across computers.

For a fuller introduction to project-based workflows, see:

https://r4ds.had.co.nz/workflow-projects.html

Objects and assignment

```
x ← 42
name ← "Anna"
passed ← TRUE
```

- `←` assigns a value to a name.
- The Environment stores the resulting objects for the current R session.
- An expression can print a result without saving it.

```
subset(whr, year == 2016)            # print only
whr2016 ← subset(whr, year == 2016) # store a new object
```

R normally does **not** modify the original object unless you assign over it.

R objects

What is an object?

- Objects are the building blocks of R programs.
- An object is something stored under a name so it can be used later.
- Objects can contain numbers, text, vectors, data frames, lists, models, plots, or functions.
- Each object has a **type**, a **class**, and a **structure** that determine how functions treat it.

```
x ← c(10, 20, 30)

typeof(x)
class(x)
str(x)
```

R objects — Object classes

Structure	Dimensions	What it can contain	Example
Vector	1	one atomic type	`c(2, 4, 6)`
Matrix	2	one atomic type	`matrix(1:6, nrow = 2)`
Data frame	2	equal-length column vectors; types may differ by column	`data.frame( ... )`
List	flexible	objects of different types and sizes	`list( ... )`

Rows normally represent observations; columns normally represent variables.

R objects — Lists and extraction

A list can contain objects of different types, classes, and lengths.

```
my_list ← list(
  name = "Anna",
  age = 20,
  scores = c(80, 90, 100)
)

my_list["scores"]     # returns a one-element list
my_list[["scores"]]   # returns the numeric vector
my_list$scores        # named extraction
```

Many complex R objects—including fitted models—are built from lists.

The central mental model of R

R objects

↓

Vectors
values of one atomic type

Data frames
variables × observations

Lists
different objects together

Object
input

↓

function()
operation

↓

Result
print it or assign it

```
result ← mean(whr$happiness_score, na.rm = TRUE)
```

Atomic data types

Type	Meaning	Example
double/numeric	ordinary real numbers	`3.14`, `10`
integer	explicitly stored whole numbers	`10L`
character	text	`"Cambridge"`
logical	truth values	`TRUE`, `FALSE`

```
typeof(10)          # "double"
typeof(10L)         # "integer"
typeof("Cambridge") # "character"
typeof(TRUE)        # "logical"
```

Vectors and type coercion

`c()` combines values into a vector:

```
scores ← c(70, 80, 90, 100)
mixed  ← c(1, 2, "Cambridge")
```

- A vector normally contains one atomic type.
- If types are mixed, R converts values to a common type.
- In `mixed`, the numbers become character strings.

```
scores[1]       # first element
scores[2:4]     # second through fourth
scores > 80     # element-wise comparison
```

R indexing starts at **1**, not 0.

Logical values and missing data

```
age ← c(18, 21, 16, NA)

age ⩾ 18
is.na(age)
mean(age, na.rm = TRUE)
```

- Comparisons return `TRUE`, `FALSE`, or sometimes `NA`.
- `==` tests equality; `←` assigns a value.
- `NA` means the value is missing or unknown.
- Many statistical functions return `NA` unless missing values are handled explicitly.

```
x ← 5   # assign 5 to x
x == 5   # ask whether x equals 5
```

Factors: categorical data

A factor stores categorical values together with a defined set of levels.

```
income_group ← factor(
  c("Low", "High", "Middle", "Low"),
  levels = c("Low", "Middle", "High"),
  ordered = TRUE
)

levels(income_group)
```

Factors matter in statistical models because their levels determine the categories, ordering, and reference group. A factor is a class built on an integer vector; it is not a separate atomic type.

Working with data frames

Data frames: variables stored as vectors

```
students ← data.frame(
  name    = c("Anna", "Ben", "Chen", "Diana"),
  age     = c(20, 22, 21, 24),
  score   = c(75, 88, 92, 65),
  country = c("UK", "US", "China", "UK")
)
```

name	age	score	country
Anna	20	75	UK
Ben	22	88	US
Chen	21	92	China
Diana	24	65	UK

A data frame is a collection of **equal-length vectors arranged as columns**.

Selecting rows, columns, and elements

The general indexing form is `data[row, column]`.

```
students[1, ]          # row 1, all columns
students[, 3]          # all rows, column 3
students[1, 3]         # one element
students[, "score"]    # a named column
students$score         # extract the score vector
students[["score"]]    # also extracts the score vector
```

- Use `$` when the column name is written directly in the code.
- Use `[[ ]]` when the column name is stored in another object.
- Remember: **row first, column second**.

Programmatic column extraction

```
column_name ← "score"

students$column_name       # NULL: looks for a column literally named "column_name"
students[[column_name]]     # extracts the score column
```

This distinction matters when a function or loop supplies the column name:

```
for (column_name in names(students)) {
  print(head(students[[column_name]], 1))
}
```

Inspecting a data frame in detail

```
head(whr)       # first 6 rows
tail(whr)       # last 6 rows
dim(whr)        # rows, columns
nrow(whr)       # number of rows
ncol(whr)       # number of columns
names(whr)      # column names
```

```
str(whr)        # compact structure
class(whr)      # object class
summary(whr)    # summaries by column
View(whr)       # spreadsheet-style viewer
```

Start with `str()` when a dataset behaves unexpectedly. It reveals the class of the object and the type of every column.

Reading the output of `str()`

```
str(students)
```

```
'data.frame': 4 obs. of 4 variables:
 $ name   : chr "Anna" "Ben" "Chen" "Diana"
 $ age    : num 20 22 21 24
 $ score  : num 75 88 92 65
 $ country: chr "UK" "US" "China" "UK"
```

This tells us:

- the object is a data frame;
- it has 4 observations and 4 variables; and
- each variable is stored as a separate vector with its own type.

Functions

Functions: input → operation → output

```
mean(whr$happiness_score, na.rm = TRUE)
```

- `mean` is the function.
- `whr$happiness_score` is the main input.
- `na.rm = TRUE` is a named argument.
- The returned value can be printed, stored, or passed to another function.

```
average_happiness ← mean(
  whr$happiness_score,
  na.rm = TRUE
)
```

Arguments, defaults, and help

```
mean(x = whr$happiness_score, na.rm = TRUE)
```

- Arguments are written inside parentheses and separated by commas.
- Named arguments make code easier to understand.
- Many arguments have defaults; for `mean()`, `na.rm` defaults to `FALSE`.

Useful ways to learn a function:

```
?mean
help(mean)
args(mean)
example(mean)
```

Nested functions and intermediate objects

These approaches produce the same final result:

```
# Intermediate object: easier to inspect and debug
log_score ← log(whr$happiness_score)
summary(log_score)

# Nested functions: compact, but can become hard to read
summary(log(whr$happiness_score))
```

Function nesting is ordinary **composition**: the inner result becomes the outer input. It is not, by itself, metaprogramming.

Data transformation

Core `dplyr` verbs in detail

```
whr2016 ← whr %>%
  filter(year == 2016) %>%                          # keep rows
  select(country, region, happiness_score) %>%     # keep/reorder columns
  mutate(score_100 = happiness_score * 10) %>%     # create/change columns
  arrange(desc(happiness_score))                   # reorder rows

regional_summary ← whr %>%
  group_by(region) %>%
  summarise(
    mean_happiness = mean(happiness_score, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

Verb	Main question
`filter()`	Which rows?
`select()`	Which columns?
`mutate()`	Which new or changed columns?
`summarise()`	Which aggregate statistics?

Pipes make a sequence explicit

The pipe passes the result on its left into the next function.

```
whr %>%
  filter(year == 2016) %>%
  select(country, happiness_score) %>%
  arrange(desc(happiness_score))
```

Read this as:

1. take `whr`;
2. keep observations from 2016;
3. retain two variables; and
4. sort from highest to lowest happiness.

Use a pipe when it makes the sequence clearer. Intermediate objects remain useful for debugging or reusing a result.

Base pipe and tidyverse pipe

Both forms are common:

```
# Base R pipe
whr |>
  head()

# magrittr/tidyverse pipe
whr %>%
  head()
```

- `|>` has been built into R since version 4.1.
- `%>%` is loaded by `dplyr` or `tidyverse` and supports some additional syntax.
- For this course, consistency within an analysis matters more than choosing one symbol over the other.

Reproducible workflows

Packages: install once, load each session

Install

```
install.packages("dplyr")
```

- downloads package files;
- normally needed once per R installation;
- package name is quoted.

Load

```
library(dplyr)
```

- attaches functions for the current session;
- repeated after restarting R;
- package name is normally unquoted.

`library(tidyverse)` loads a collection that includes `dplyr`, `ggplot2`, `tidyr`, `readr`, and `purrr`.

Errors and warnings

- An **error** stops the current expression because R cannot complete it.
- A **warning** reports a possible problem even though the expression completed.
- A **message** provides information and is not necessarily a problem.

When something fails:

1. read the first error message carefully;
2. check object and column names;
3. inspect the object with `str()`;
4. run the code in smaller pieces; and
5. check the function documentation.

A reproducible script header

```r
# 1. Load packages ----------------------------------------
library(tidyverse)
library(here)

# 2. Import data ------------------------------------------
whr ← read.csv(
  here("DataWork", "DataSets", "Final", "whr_panel.csv")
)

# 3. Inspect ----------------------------------------------
str(whr)

# 4. Transform --------------------------------------------
whr2016 ← whr %>%
  filter(year == 2016)
```

Avoid depending on objects left over from a previous session.

Key takeaways

- R stores values, data, models, and functions as objects.
- A data frame is built from equal-length column vectors.
- Inspect unfamiliar data with str(), head(), dim(), and summary().
- Functions take arguments and return results; assignment stores those results.
- Use filter(), select(), mutate(), and summarise() for core data work.
- Projects, portable paths, and scripts make an analysis reproducible.

Thank you!

Appendix

Appendix - R vs Stata

- R is object oriented while Stata is action oriented:
  - Classic example: Stata's `summarize` vs R's `summary()`
  - In Stata you declare what you want to do, while in R you usually declare the result you want to get
- R needs to load non-base commands (packages) at the beginning of each session
  - Imagine that in Stata you'd have to load a command installed with `ssc install` every time you'll use it in a new session
- R is less specialized, which means more flexibility and functionalities.
- R has a much broader network of users:
  - More resources online, which makes using Google a lot easier. You'll never want to see Statalist again in your life!
  - Development of new features and bug fixes happen faster.

Appendix - Syntax

R's syntax is heavier than Stata's:

- Parentheses to separate function names from its arguments.
- Commas to separate arguments.
- For comments we use the `#` sign.
- You can have line breaks inside function statements.
- In R, functions can be treated much like any other object. Therefore, they can be passed as arguments to other functions.

Similarly to Stata:

- Square brackets are used for indexing.
- Curly braces are used for loops and if statements.
- Largely ignores white spaces.

Appendix - Help, Google and Stack Overflow

Help in R works very much like in Stata: the help files usually start with a brief description of the function, explain its syntax and arguments and list a few examples. There are two ways to access help files:

Exercise 7: Use help

```
# You can use the help() function
help(summary)

# or its abbreviation
?summary
```

Appendix - Help, Google and Stack Overflow

- The biggest difference, however, is that **R has a much wider user community** and it has **a lot more online resources**.
- For instance, by 2014, Stata had 11 dedicated blogs written by users, while R had 550 (check http://r4stats.com/articles/popularity/ for more details).
- The most powerful problem-solving tool in R, however, is Google. Searching the something yields tons of results.
- Often that means a Stack Overflow page where someone asked the same question and several people gave different answers. Here's a typical example: https://stackoverflow.com/questions/1660124/how-to-sum-a-variable-by-group
- You can also ask ChatGPT for help in R. But be careful as it can sometimes hallucinate or give your incorrect answers.

Appendix - Useful resources

Blogs, courses and resources:

- Surviving graduate econometrics with R: https://thetarzan.wordpress.com/2011/05/24/surviving-graduate-econometrics-with-r-the-basics-1-of-8/
- CRAN's manuals: https://cran.r-project.org/manuals.html
- R programming in Coursera: https://www.coursera.org/learn/r-programming
- R programming for dummies: http://www.dummies.com/programming/r/
- R bloggers: https://www.r-bloggers.com/
- R statistics blog: https://www.r-statistics.com/
- The R graph gallery: https://www.r-graph-gallery.com/

Appendix - Useful resources

Books:

- R for Stata Users - Robert A. Muenchen and Joseph Hilbe https://link.springer.com/book/10.1007/978-1-4419-1318-0
- R Graphics Cookbook - Winston Chang https://r-graphics.org/
- R for Data Science - Hadley Wickham, Mine Çetinkaya-Rundel, and Garrett Grolemund https://r4ds.hadley.nz//
- Advanced R - Hadley Wickham https://adv-r.hadley.nz/

Appendix - More on packages

Once a package is loaded, you can use its functions. Some useful examples include:

- `swirl`: interactive learning for R and statistics
- `ggplot2`: data visualization
- `data.table`: efficient work with large data frames
- `zoo`: time-series tools
- `sandwich`: robust covariance estimators
- `sf`: modern spatial data analysis
- `DBI`, `RSQLite`, and `RPostgres`: database connections

Choose packages according to the analytical task and check that their documentation is current.